Introducción a la Ingeniería de Datos

Maestría en Inteligencia Artificial - MIA 03

Facultad Politécnica - Universidad Nacional de Asunción

Clase 7: Orquestación y Visualización

Contenidos

Caso de Uso: Maven Fuzzy Factory 3
Paso 1: Configurar Airbyte 8
Paso 2: Modelos dbt 14
Paso 3: Orquestación con Prefect 27
Paso 4: Visualización con Metabase 41
Actividad Asincrónica 54

Caso de Uso: Maven Fuzzy Factory

Dataset: Ecommerce de Juguetes

Maven Fuzzy Factory

Base de datos de un ecommerce ficticio que vende juguetes. Contiene datos reales de:

- Sesiones web y navegación
- Campañas de marketing (UTM)
- Órdenes y productos
- Reembolsos

Dataset: Ecommerce de Juguetes

Ideal para análisis de negocio

- Conversion rate por canal
- Revenue y margen por producto
- Análisis de funnel
- Comportamiento de usuarios

Modelo de Datos

Tabla	Descripción
`website_sessions`	Sesiones de usuarios (UTM, device, referer)
`website_pageviews`	Páginas visitadas por sesión
`products`	Catálogo de productos
`orders`	Órdenes realizadas
`order_items`	Items dentro de cada orden
`order_item_refunds`	Reembolsos procesados

450K sesiones, 32K órdenes, 4 productos

Arquitectura del Pipeline

MySQL - Base de datos transaccional

↓

Airbyte - Extract & Load

↓

MotherDuck - Data Warehouse

↓

dbt - Transform

↓

Metabase - Visualize

Paso 1: Configurar Airbyte

Crear Source: MySQL

En Airbyte UI

1. **Sources → New Source**
2. Buscar «MySQL»
3. Configurar:

Campo	Valor
Host	localhost o IP del servidor
Port	3306
Database	maven_fuzzy_factory
Username	tu_usuario
Password	tu_password

Crear Connection MySQL → MotherDuck

Configurar Connection

1. **Connections → New Connection**
2. Source: MySQL (recién creado)
3. Destination: MotherDuck (database: `airbyte_curso`)
4. Seleccionar las 6 tablas a sincronizar

Crear Connection MySQL → MotherDuck

Sync settings

- Sync mode: **Full refresh | Overwrite** (datos históricos)
- Schedule: **Manual** (lo orquestaremos con Prefect)
- Destination namespace: **Custom** → `main`

Seleccionar Tablas

Tabla	Sync	Registros
website_sessions	✓	450K
website_pageviews	✓	1.1M
orders	✓	32K
order_items	✓	54K
order_item_refunds	✓	800
products	✓	4

Ejecutar Primera Sincronización

Verificar datos

1. Click en **Sync now** en la connection
2. Esperar a que complete (2-5 minutos)
3. Verificar en MotherDuck:

```
USE airbyte_curso;
SELECT 'orders' as t, COUNT(*) as n FROM main.orders
UNION ALL SELECT 'sessions', COUNT(*) FROM main.website_sessions;
```

Paso 2: Modelos dbt

Estructura del Proyecto dbt

Archivos a crear

```
models/
├── staging/
│   ├── _sources.yml
│   ├── stg_sessions.sql
│   ├── stg_orders.sql
│   └── stg_order_items.sql
└── marts/
    ├── fct_daily_sales.sql
    ├── fct_channel_performance.sql
    └── obt_orders_enriched.sql
```

Estructura del Proyecto dbt

Los modelos se crean en schemas `main_staging` y `main_marts`

Sources: Tablas desde Airbyte

models/staging/_sources.yml

```yaml
version: 2

sources:
  - name: raw
    database: airbyte_curso
    schema: main
    tables:
      - name: website_sessions
      - name: website_pageviews
      - name: orders
      - name: order_items
      - name: order_item_refunds
      - name: products
```

Staging: Sessions

models/staging/stg_sessions.sql

```sql
with source as (
    select * from {{ source('raw', 'website_sessions') }}
)

select
    website_session_id,
    created_at,
    user_id,
    is_repeat_session = 1 as is_repeat_session,
    coalesce(utm_source, 'direct') as utm_source,
    coalesce(utm_campaign, 'none') as utm_campaign,
    utm_content,
    device_type,
    http_referer,
    date_trunc('day', created_at) as session_date,
    date_trunc('month', created_at) as session_month
from source
```

Staging: Orders

models/staging/stg_orders.sql

```sql
with source as (
    select * from {{ source('raw', 'orders') }}
)

select
    order_id,
    created_at,
    website_session_id,
    user_id,
    primary_product_id,
    items_purchased,
    price_usd,
    cogs_usd,
    price_usd - cogs_usd as margin_usd,
    date_trunc('day', created_at) as order_date,
    date_trunc('month', created_at) as order_month
from source
```

Staging: Order Items

models/staging/stg_order_items.sql

```sql
with source as (
    select * from {{ source('raw', 'order_items') }}
),

products as (
    select * from {{ source('raw', 'products') }}
)

select
    oi.order_item_id,
    oi.order_id,
    oi.product_id,
    p.product_name,
    oi.is_primary_item = 1 as is_primary_item,
    oi.price_usd,
    oi.cogs_usd,
    oi.price_usd - oi.cogs_usd as margin_usd
from source oi
left join products p on oi.product_id = p.product_id
```

Mart: Ventas Diarias

models/marts/fct_daily_sales.sql

```sql
{{ config(materialized='table') }}

with sessions as (
    select session_date, count(*) as total_sessions
    from {{ ref('stg_sessions') }}
    group by session_date
),
orders as (
    select order_date, count(*) as total_orders,
           sum(price_usd) as revenue, sum(margin_usd) as margin
    from {{ ref('stg_orders') }}
    group by order_date
)

select
    s.session_date as date,
    s.total_sessions,
    coalesce(o.total_orders, 0) as total_orders,
    round(o.total_orders * 100.0 / s.total_sessions, 2) as conversion_rate,
    coalesce(o.revenue, 0) as revenue,
    coalesce(o.margin, 0) as margin
from sessions s
left join orders o on s.session_date = o.order_date
```

Mart: Performance por Canal

models/marts/fct_channel_performance.sql

```sql
{{ config(materialized='table') }}

with sessions as (select * from {{ ref('stg_sessions') }}),
orders as (select * from {{ ref('stg_orders') }})

select
    s.session_month,
    s.utm_source,
    s.utm_campaign,
    s.device_type,
    count(distinct s.website_session_id) as sessions,
    count(distinct o.order_id) as orders,
    round(count(distinct o.order_id) * 100.0 / count(distinct s.website_session_id), 2) as conversion_rate,
    coalesce(sum(o.price_usd), 0) as revenue,
    round(coalesce(sum(o.price_usd), 0) / count(distinct s.website_session_id), 2) as revenue_per_session
from sessions s
left join orders o on s.website_session_id = o.website_session_id
group by s.session_month, s.utm_source, s.utm_campaign, s.device_type
```

Mart: OBT Orders Enriched

models/marts/obt_orders_enriched.sql

```sql
{{ config(materialized='table') }}

with orders as (select * from {{ ref('stg_orders') }}),
sessions as (select * from {{ ref('stg_sessions') }}),
products as (select * from {{ source('raw', 'products') }})

select
    o.order_id, o.created_at, o.order_date, o.order_month,
    o.user_id, o.items_purchased, o.price_usd, o.cogs_usd, o.margin_usd,
    s.utm_source, s.utm_campaign, s.device_type, s.is_repeat_session,
    p.product_name as primary_product_name,
    case
        when o.price_usd >= 100 then 'High Value'
        when o.price_usd >= 50 then 'Medium Value'
        else 'Low Value'
    end as order_tier
from orders o
left join sessions s on o.website_session_id = s.website_session_id
left join products p on o.primary_product_id = p.product_id
```

Ejecutar dbt

Comandos

```
# Ejecutar todos los modelos
dbt run

# Ejecutar tests
dbt test

# Ver documentación
dbt docs generate && dbt docs serve
```

Ejecutar dbt

Resultado: tablas en `main_staging` y `main_marts`

☕ Break

15 minutos

Regresamos para orquestación y visualización

Paso 3: Orquestación con Prefect

¿Por Qué Orquestar?

Sin orquestación

```
# Ejecutar manualmente cada paso
# 1. Click en Airbyte UI
# 2. Esperar...
# 3. dbt run && dbt test
```

Problemas: ¿Quién ejecuta a las 3am? ¿Qué pasa si falla? ¿Cómo monitoreamos?

Instalación de Prefect y Dependencias

Instalar paquetes

```
pip install prefect prefect-dbt httpx python-dotenv

# Verificar
prefect version

# Iniciar servidor local (opcional)
prefect server start  # UI en http://localhost:4200
```

Nota: Usamos `httpx` para llamar a la API de Airbyte directamente.

Conceptos: Tasks y Flows

Task - Unidad de trabajo

```python
from prefect import task

@task
def extract():
    print("Extrayendo...")
    return data
```

Flow - Orquesta tasks

```python
from prefect import flow

@flow
def pipeline():
    data = extract()
    transform(data)
```

Integración con Airbyte via API

Airbyte expone una API REST con autenticación

```python
import httpx

base_url = "http://localhost:8000/api/v1"

with httpx.Client(auth=("airbyte", "password")) as client:
    # Trigger sync
    response = client.post(
        f"{base_url}/connections/sync",
        json={"connectionId": "tu-connection-id"}
    )
    job_id = response.json()["job"]["id"]

    # Verificar estado (POST, no GET)
    status = client.post(f"{base_url}/jobs/get", json={"id": job_id})
```

¿Dónde Obtener el Connection ID?

En la UI de Airbyte

1. Ir a **Connections**
2. Click en tu conexión MySQL → MotherDuck
3. El ID está en la URL:

```
http://localhost:8000/.../connections/e1b2078f-882a-...
```

También disponible en **Settings** dentro de la conexión.

Task: Extraer y Cargar con Airbyte

Usando httpx con autenticación

```python
@task(name="Extract and Load", retries=2, retry_delay_seconds=60)
def extract_and_load():
    logger = get_run_logger()
    base_url = f"http://{AIRBYTE_HOST}:{AIRBYTE_PORT}/api/v1"

    with httpx.Client(timeout=30, auth=(AIRBYTE_USERNAME, AIRBYTE_PASSWORD)) as client:
        response = client.post(
            f"{base_url}/connections/sync",
            json={"connectionId": AIRBYTE_CONNECTION_ID}
        )
        if response.status_code == 409:  # Sync ya en curso
            logger.warning("Ya hay un sync en curso, esperando...")
            jobs_resp = client.post(f"{base_url}/jobs/list",
                json={"configTypes": ["sync"], "configId": AIRBYTE_CONNECTION_ID})
            job_id = jobs_resp.json()["jobs"][0]["job"]["id"]
        else:
            response.raise_for_status()
            job_id = response.json()["job"]["id"]

        while True:  # Polling
            status_resp = client.post(f"{base_url}/jobs/get", json={"id": job_id})
            status = status_resp.json()["job"]["status"]
            if status == "succeeded": return job_id
            elif status in ("failed", "cancelled"): raise RuntimeError(f"Job falló: {status}")
            time.sleep(10)
```

Task: Transformar con dbt

Usando prefect-dbt

```python
from typing import Optional
from prefect_dbt.cli.commands import DbtCoreOperation

@task(name="Transform with dbt")
def transform(select: Optional[str] = None):
    commands = ["dbt deps"]
    if select:
        commands.append(f"dbt run --select {select}")
    else:
        commands.append("dbt run")

    DbtCoreOperation(
        commands=commands,
        project_dir=str(DBT_PROJECT_DIR),
        profiles_dir=str(DBT_PROFILES_DIR)
    ).run()
```

Configuración del Pipeline

Archivo .env

```
AIRBYTE_HOST=localhost
AIRBYTE_PORT=8000
AIRBYTE_CONNECTION_ID=tu-connection-id
AIRBYTE_USERNAME=airbyte
AIRBYTE_PASSWORD=password
```

Cargar en Python

```python
from dotenv import load_dotenv
load_dotenv()

AIRBYTE_HOST = os.getenv("AIRBYTE_HOST", "localhost")
AIRBYTE_CONNECTION_ID = os.getenv("AIRBYTE_CONNECTION_ID")
```

Flow: Pipeline Completo

ecommerce_pipeline.py

```python
@flow(name="Ecommerce ELT Pipeline")
def ecommerce_pipeline(
    run_extract: bool = True,
    run_transform: bool = True,
    run_tests: bool = True,
    run_docs: bool = False,
    dbt_select: Optional[str] = None
):
    logger = get_run_logger()
    logger.info("Iniciando pipeline ELT de Maven Fuzzy Factory")

    if run_extract:
        extract_and_load()

    if run_transform:
        transform(select=dbt_select)

    if run_tests:
        test_data(select=dbt_select)

    if run_docs:
        generate_docs()

    logger.info("Pipeline completado!")
    return {"status": "success"}
```

Deployment con schedule

```python
if __name__ == "__main__":
    ecommerce_pipeline.serve(
        name="ecommerce-daily",
        cron="0 6 * * *",  # Diario a las 6am
        parameters={
            "run_extract": True,
            "run_transform": True,
            "run_tests": True,
            "run_docs": False
        }
    )
```

Programar Ejecución Automática

Cron expressions

`0 6 * * *` diario 6am | `0 */4 * * *` cada 4 horas | `0 6 * * 1` lunes 6am

Pipeline Solo dbt (sin Airbyte)

Para ejecutar solo transformaciones

```python
@flow(name="dbt Only Pipeline")
def dbt_only_pipeline(
    select: Optional[str] = None,
    run_tests: bool = True
):
    transform(select=select)
    if run_tests:
        test_data(select=select)
```

Útil para desarrollo: solo transformar sin extraer datos nuevamente

Paso 4: Visualización con Metabase

Crear Dockerfile con driver DuckDB

```
FROM eclipse-temurin:21-jre

ENV MB_PLUGINS_DIR=/plugins
RUN mkdir -p ${MB_PLUGINS_DIR} /app

# Metabase v0.58.8
ADD https://downloads.metabase.com/v0.58.8/metabase.jar /app/metabase.jar

# DuckDB driver
ADD https://github.com/motherduckdb/metabase_duckdb_driver/releases/\
download/1.4.3.1/duckdb.metabase-driver.jar ${MB_PLUGINS_DIR}/

EXPOSE 3000
CMD ["java", "-jar", "/app/metabase.jar"]
```

Construir y Ejecutar

Comandos

```
# Construir imagen
docker build -f Dockerfile.metabase -t metabase-duckdb:latest .

# Ejecutar
docker run -d -p 3000:3000 \
  --name metabase \
  -v metabase-data:/metabase.db \
  -e MB_PLUGINS_DIR=/plugins \
  metabase-duckdb:latest
```

Construir y Ejecutar

Abrir http://localhost:3000 y completar setup inicial

Conectar a MotherDuck

Admin → Databases → Add database

Campo	Valor
Database type	DuckDB
Display name	Ecommerce DWH
Database file	`md:airbyte_curso`
MotherDuck Token	**(campo separado)**

Conectar a MotherDuck

El token de MotherDuck se ingresa en su propio campo, no en el nombre de la BD

Visualización 1: KPIs Principales

SQL Query

```sql
SELECT
  COUNT(*) AS total_orders,
  ROUND(SUM(price_usd), 2) AS total_revenue,
  ROUND(SUM(margin_usd), 2) AS total_margin,
  ROUND(AVG(price_usd), 2) AS avg_order_value
FROM main_marts.obt_orders_enriched
```

Crear 4 **Number** cards: Total Orders, Revenue, Margin, AOV

Visualización 2: Revenue por Mes

SQL Query

```sql
SELECT
  order_month,
  ROUND(SUM(price_usd), 2) AS revenue,
  ROUND(SUM(margin_usd), 2) AS margin,
  COUNT(*) AS orders
FROM main_marts.obt_orders_enriched
GROUP BY order_month
ORDER BY order_month
```

Visualización: **Line chart** o **Bar chart** con tendencia mensual

Visualización 3: Conversion por Canal

SQL Query

```sql
SELECT
  utm_source,
  utm_campaign,
  sessions,
  orders,
  conversion_rate,
  revenue
FROM main_marts.fct_channel_performance
WHERE session_month = (
  SELECT MAX(session_month) FROM main_marts.fct_channel_performance
)
ORDER BY revenue DESC
```

Visualización: **Table** con formato condicional en conversion_rate

Visualización 4: Revenue por Producto

SQL Query

```sql
SELECT
  primary_product_name,
  COUNT(*) AS orders,
  ROUND(SUM(price_usd), 2) AS revenue,
  ROUND(AVG(price_usd), 2) AS avg_price
FROM main_marts.obt_orders_enriched
GROUP BY primary_product_name
ORDER BY revenue DESC
```

Visualización: **Pie chart** o **Bar chart** horizontal

Visualización 5: Ventas Diarias

SQL Query

```sql
SELECT
  date,
  total_sessions,
  total_orders,
  conversion_rate,
  revenue
FROM main_marts.fct_daily_sales
ORDER BY date DESC
LIMIT 30
```

Visualización: **Line chart** con últimos 30 días

Dashboard Layout

Orders Number	Revenue Number	Margin Number	AOV Number

Revenue por Mes - Line chart

Por Producto Pie	Ventas Diarias Line

Performance por Canal - Table

Agregar Filtros

Filtros recomendados

1. **Date range:** Filtrar por `order_month` u `order_date`
2. **UTM Source:** Filtrar por canal de adquisición
3. **Device type:** Desktop vs Mobile
4. **Product:** Filtrar por producto específico

En modo edición → **Filter** → Mapear a columnas de cada card

Actividad Asincrónica

Tarea Práctica - Clase 7

Entregables

1. **Airbyte:** Connection MySQL → MotherDuck funcionando
2. **dbt:** Modelos en `main_staging` y `main_marts` ejecutándose
3. **Prefect:** Pipeline completo con archivo `.env` configurado
4. **Metabase:** Dashboard con al menos 5 visualizaciones y 2 filtros
5. **Capturas:** Prefect UI mostrando ejecución exitosa + Dashboard

Foro: Del Dato al Insight

Consigna

Compartí una **captura de tu dashboard** y respondé:

1. ¿Qué **preguntas de negocio** responden tus visualizaciones?
2. ¿Qué **insights** encontraste en los datos? (ej: mejor canal, producto más rentable)
3. ¿Qué **métricas adicionales** agregarías para un gerente de ecommerce?

Comentá el dashboard de un compañero sugiriendo una mejora.

¿Preguntas?

📧 Contacto del docente

📚 Material en plataforma

💬 Foro de consultas abierto

Resumen Clase 7

Pipeline completo

1. MySQL → Airbyte → MotherDuck
2. dbt: `main_staging` + `main_marts`
3. Prefect: orquestación con httpx
4. Metabase: dashboard interactivo

Métricas de negocio

- Conversion rate por canal
- Revenue y margin
- Performance por producto
- Ventas diarias

Próxima clase: Proyecto Final - Presentaciones